

ONNX Runtime for Edge Models

Portable, optimized inference across devices



What Is ONNX?



Open Standard

Open Neural Network Exchange (ONNX) is an open format to represent deep learning models



Framework Support

Supported by PyTorch, TensorFlow, Scikit-learn, and Hugging Face



Interoperability

Train in one framework → deploy anywhere

ONNX is specifically designed to be [lightweight and optimized for edge inference](#), making it ideal for resource-constrained environments.

ONNX Runtime Overview



Cross-platform inference engine maintained by Microsoft that runs on:

- **x86, ARM processors**
- **GPUs, NPU, Jetson boards**
- **Mobile and embedded devices**

Key Features

- Hardware acceleration (CUDA, TensorRT, DirectML)
- Quantization for efficiency
- Cross-language APIs: Python, C++, Java, C#

Why ONNX for Edge AI?

Portability

One model runs across multiple hardware platforms without modification

Performance

Optimized kernels for CPU/GPU/TPU with hardware-specific acceleration

Flexibility

Supports both deep learning and classical ML models in a single format

Compatibility

Integrates with TensorRT, OpenVINO, Core ML and other edge platforms

The combination of these advantages makes ONNX an ideal choice for deployments where consistent performance across heterogeneous devices is critical.

Exporting Models to ONNX

Converting your trained PyTorch model to ONNX is straightforward:

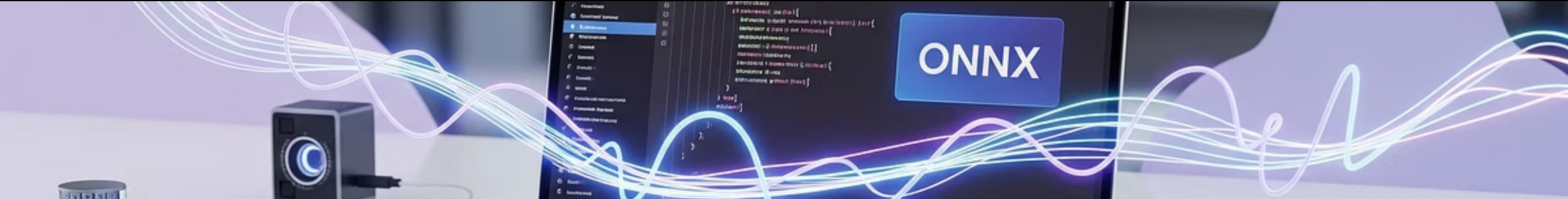
```
import torch
import torchvision.models as models

model = models.resnet18(pretrained=True)
dummy_input = torch.randn(1, 3, 224, 224)

torch.onnx.export(model,
                  dummy_input,
                  "resnet18.onnx",
                  input_names=["input"],
                  output_names=["output"])
```

Result: `resnet18.onnx` portable model file that can be deployed across platforms.

-  The dummy input tensor shapes must match your expected inference inputs. For image models, this is typically [batch_size, channels, height, width].



Running with ONNX Runtime (Python)

```
import onnxruntime as ort
import numpy as np

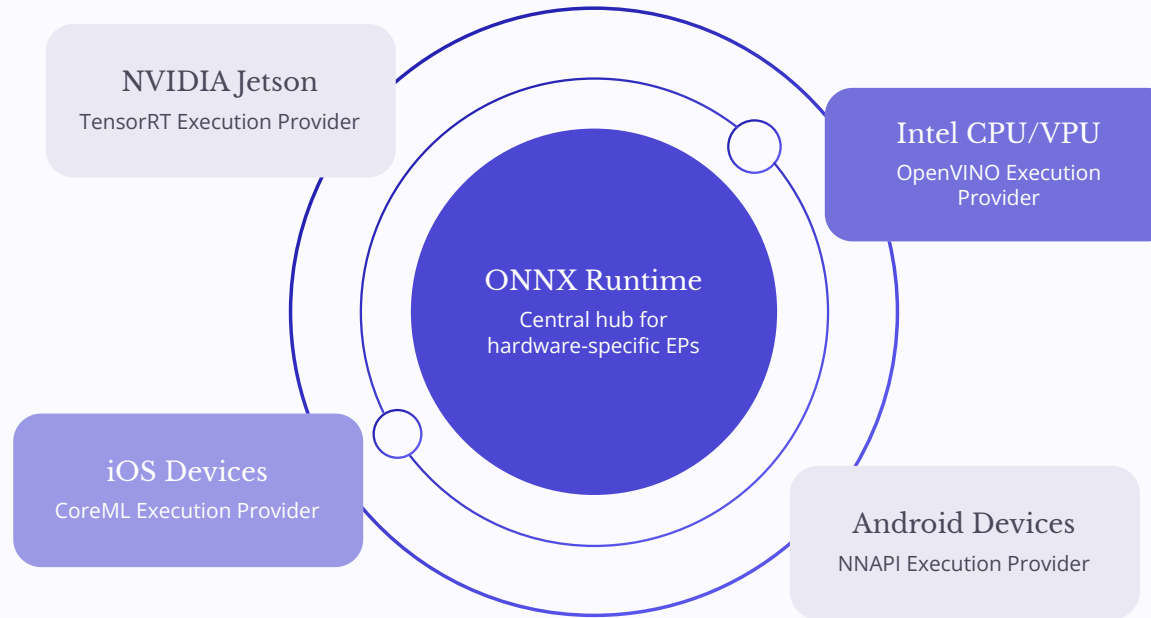
# Load model
session = ort.InferenceSession("resnet18.onnx")

# Prepare input
inputs = {session.get_inputs()[0].name:
          np.random.rand(1, 3, 224, 224).astype(np.float32)}

# Run inference
outputs = session.run(None, inputs)
print(outputs[0].shape) # predictions
```

- ❑ ONNX Runtime automatically selects the best available execution provider for your hardware. You can also explicitly specify execution providers in order of preference.

Edge Hardware Acceleration



The same ONNX model works across environments, providing [cloud-to-edge parity](#) with optimized performance through hardware-specific Execution Providers (EPs).

ONNX Optimizations

Quantization

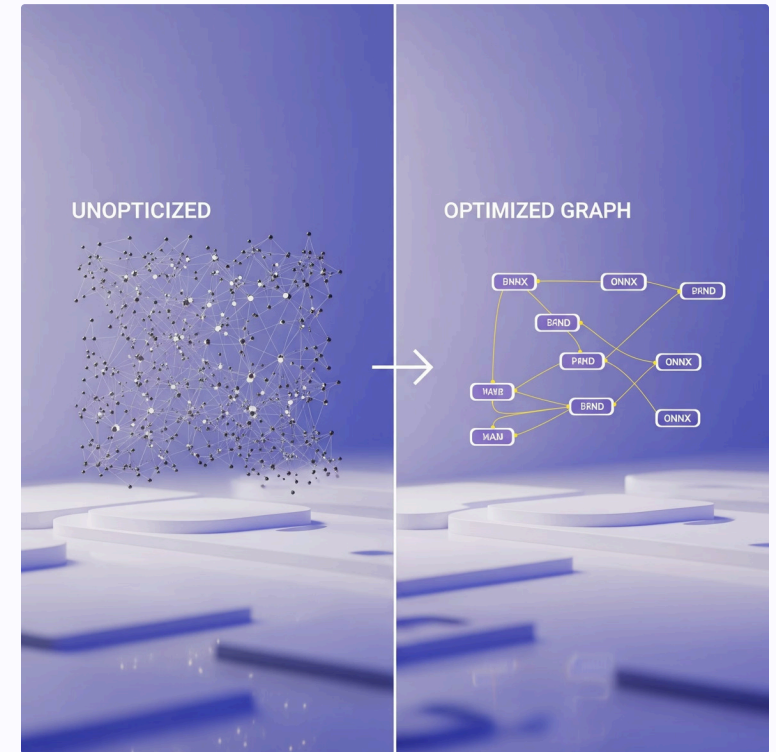
INT8, FP16 conversion for smaller, faster models with minimal accuracy loss

Graph Optimization

Fuse operations, eliminate redundancies, remove unused nodes

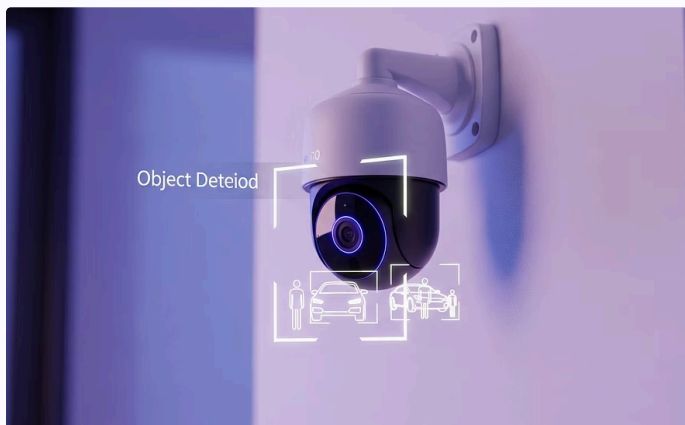
Execution Providers

Runtime selects best backend based on available hardware



ONNX Runtime's optimizations are crucial for **real-time inference at the edge**, where computational resources and power are limited.

Real-World Use Cases



Computer Vision

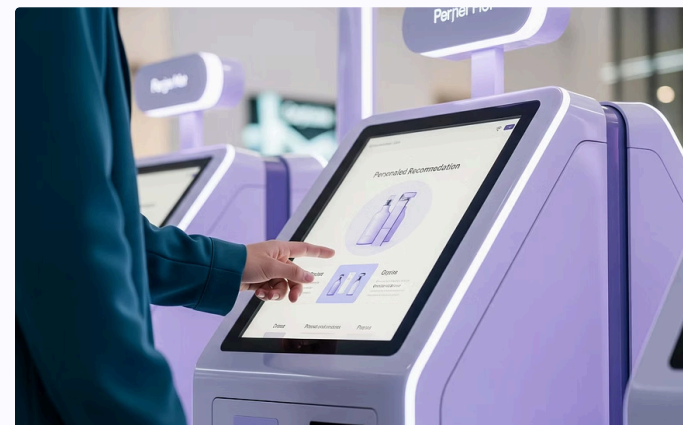
Smart cameras running YOLOv5 exported to ONNX for object detection and surveillance

These applications benefit from ONNX's [optimized edge performance](#) while maintaining deployment flexibility across diverse hardware.



Predictive Maintenance

ML pipelines in factories analyzing sensor data to predict equipment failures



Retail Intelligence

Real-time personalization at point of sale using on-device inference

Key Takeaways

- ONNX provides a **universal model format** for true portability
- ONNX Runtime delivers optimized inference for both edge and cloud
- Hardware acceleration and quantization enable efficient deployment
- Essential for scaling AI across heterogeneous device fleets

✅ Start today: `pip install onnx onnxruntime`

